Qualité de développement

Nicolas Ferry
Université Côte d'Azur – IUT Informatique – BUT 1
2025 - 2026

Savoirs de référence à étudier

L'objectif de cette ressource est de vous initier à la qualité de développement.

Savoirs faire à étudier:

- Première approche de la gestion des cas d'erreurs (ex : Exception,...)
- Sensibilisation à la production de tests unitaires, problématique de la non-régression
- Automatisation de tests unitaires
- Traces et utilisation d'outils de débogage
- Utilisation d'un outil de gestion de versions

Prérequis et liens avec les SAEs

- **Prérequis :**
  - D'après le PPN aucun !
  - Dans les faits R2.01 !

- **SAEs:**
  - S2.01 – Dev d'application
  - S2.02 – Exploration algo
  - S2.05 – Gestion de projet

Agenda

- TD: 10 séances de 2 heures pour un total de 20 heures.
- CM: 4 heures de cours

- **Séance 1:** Clean code *(Cours GIT + Clean Code)*
- **Séance 2:** Git
- **Séance 2:** Git *(Cours Tests)*
- **Séance 4:** Tests unitaires + *Contrôle Git*
- **Séance 5:** Tests unitaires
- **Séance 6:** Mocks
- **Séance 7:** Trace d'exécution et outils de log + *Contrôle Tests*
- **Séance 8:** Trace d'exécution et outils de log
- **Séance 9:** Debugger
- *DS Final*

Evaluation

- La ressource est évaluée via :
  - Un contrôle final.
  - Des petits contrôles intermédiaires (x2).
    - Pendant les TPs.

Qualité de développement

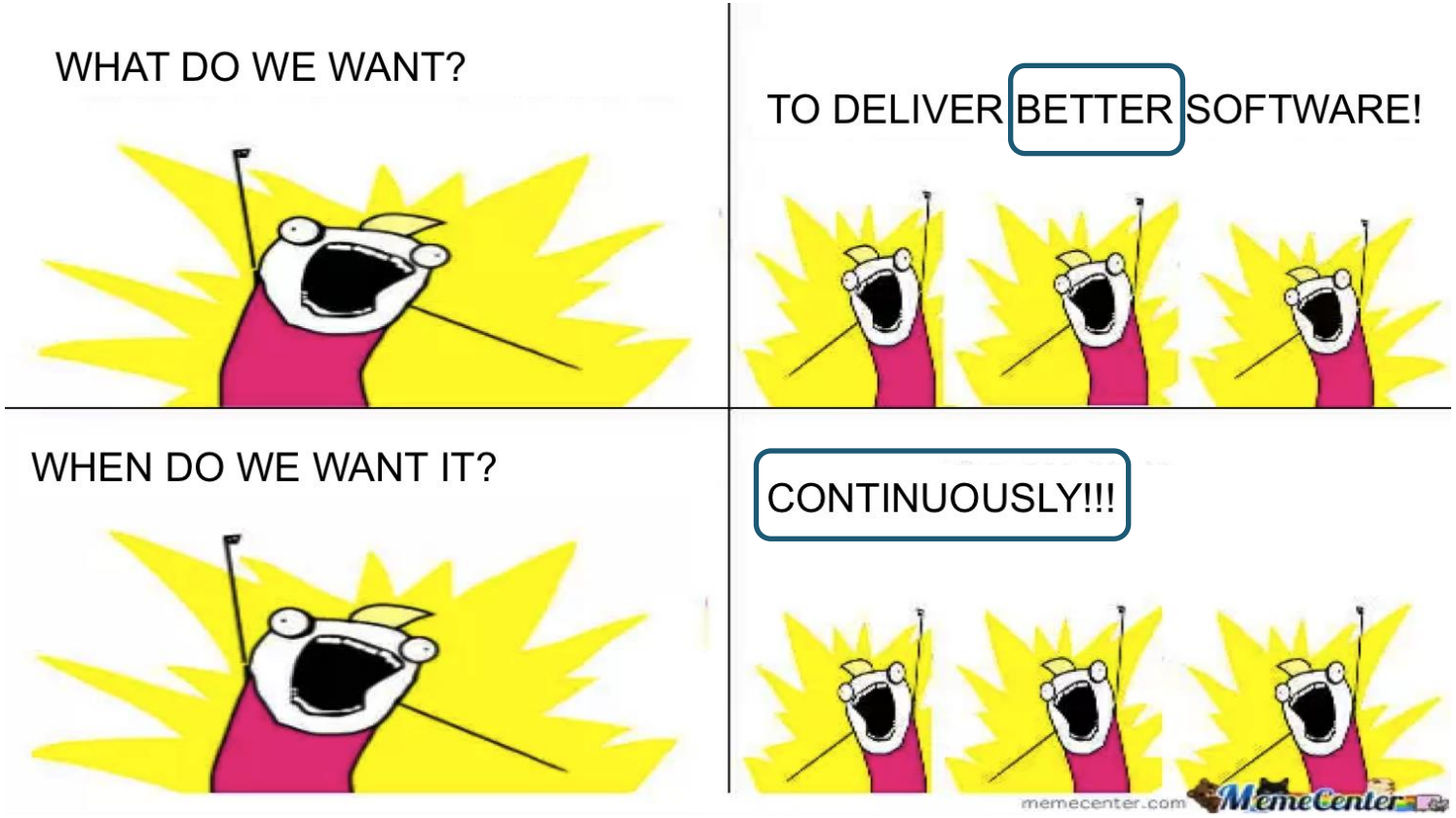
- On parle de quoi ?
  - Améliorer la lisibilité et la réutilisabilité du code
  - Faciliter la collaboration, traçabilité, etc.
  - Vérifier le code avant de le livrer
  - Valider le code avant de le livrer
  - …

Qualité de développement – *Pourquoi ?*

- Pour s’adapter plus facilement au changement.
- Pour partager plus facilement son code / collaborer.
- Pour améliorer la qualité du logiciel et la confiance dans son logiciel.
- …

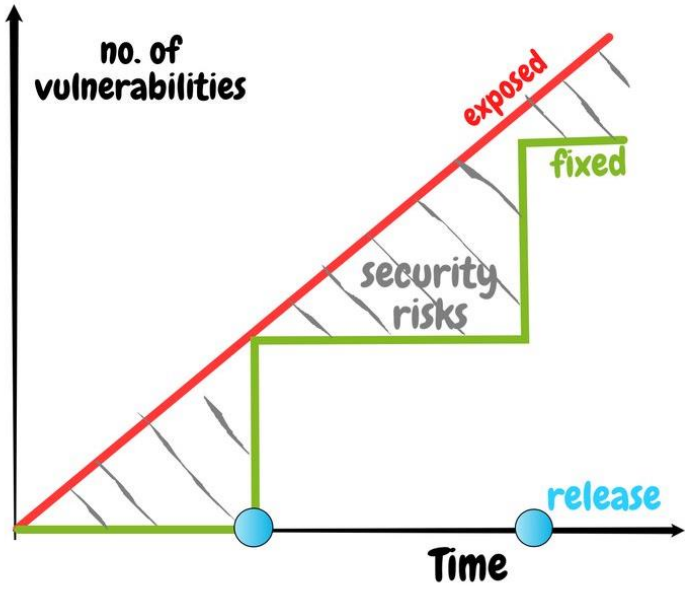
Qualité de développement – *Pourquoi ?*

- Mon but en tant que fournisseur de logiciel :



Pourquoi

- Pour pouvoir **livrer** et **maintenir (la qualité)** de manière continue, rapide, et incrémentale :
  - Fournir le plus rapidement les fonctionnalités aux clients.
  - Mise à jour (chgt de besoins, bugs, failles de sécurité, etc.) plus rapide et réactives.
  - Réduire les coûts (parfois, souvent).
  - Forcer, en automatisant les tests, la qualité



Observer et analyser

- La condition *sine qua non* pour pouvoir automatiser le processus de livraison / déploiement : **Confiance**
  1. Avoir confiance en son processus / mécanisme de livraison.
  2. Que les clients aient confiance dans les livraisons (i.e., si une fois sur deux ce que je livre ne marche pas …)

- Pour construire la confiance:
  - Tests
  - Répétabilité du processus
  - Analyse du risque
  - Traçabilité
  - …

Clean Code

A part of it!

Clean Code

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

-- Martin Fowler

« Clean code is simple and direct. Clean code reads like well-written prose. Clean code never obscures the designer's intent but rather is full of crisp abstractions and straightforward lines of control »

-- Grady Booch

Une série de principes pour produire un code **compréhensible de façon intuitive** et **facile à modifier**.

Clean code

Ce que l'on ne veut pas !

```
223        getScoreFor(3),
224        getScoreFor(4),
225        getScoreFor(5),
226        getScoreFor(6),
227        getScoreFor(7),
228        getScoreFor(8),
229        getScoreFor(9),
230        getScoreFor(10),
231        getScoreFor(11),
232        getScoreFor(12),
233        getScoreFor(13),
234        getScoreFor(14),
235        getScoreFor(15),
236      ]).then((response) => {
237        // console.log("Réponse du tableau : ", response);
238        if (
239          isNaN(response[0]) == false &&
240          isNaN(response[1]) == false &&
241          isNaN(response[2]) == false &&
242          isNaN(response[3]) == false &&
243          isNaN(response[4]) == false &&
244          isNaN(response[5]) == false &&
245          isNaN(response[6]) == false &&
246          isNaN(response[7]) == false &&
247          isNaN(response[8]) == false &&
248          isNaN(response[9]) == false &&
249          isNaN(response[10]) == false &&
250          isNaN(response[11]) == false &&
251          isNaN(response[12]) == false &&
252          isNaN(response[13]) == false &&
253          isNaN(response[14]) == false
254        ) {
255          const finalTableSorted = createFinalTable(
256            response[0],
257            response[1],
258            response[2],
259            response[3],
260            response[4],
261            response[5],
262            response[6],
263            response[7],
264            response[8],
265            response[9],
266            response[10],
267            response[11],
268            response[12],
269            response[13],
```


UNIVERSITÉ CÔTE D'AZUR
iut Nice Côte d'Azur

Ce qui doit vous faire réfléchir

- Suivi des conventions ;
- Complexité du code ;
- Code dupliqué ;
- Méthode trop longue ;
- Méthode avec plusieurs intentions ;
- Noms non parlants ;
- Nombres magiques ;
- Non utilisation de constantes ;
- Classe trop longue ;
- Méthode avec trop de paramètres en entrée ;

Nommage

- Les **noms doivent** :
  - Révéler **l'intention**.
  - Avoir du **sens**.
  - Etre **prononçables** (attention aux abréviations !).
  - *Astuce: est-ce que j'utiliserais ce mot pour chercher cette méthode dans mon code ?*
- Les classes doivent être des noms
  - Attention informations redondantes : User.getUserInfo() …
- Des verbes pour les méthodes / fonctions : **get**XXX(), **set**XXX(), **create**XXX(), **validate**XXX()

Commentaires

- **Les commentaires ne sont pas une excuse pour laisser du code sale ou incompréhensible !!!!**
  - ~~Mon code est moche pas grave je l'explique avec un commentaire …~~
  - Si mon code est incompréhensible alors je **refactor** pour le rendre compréhensible !!

- Ils mentent, vieillissent mal, ne sont pas facilement refactorables.

- A privilégier pour des explications (Javadoc, algorithmes mathématiques, contournement de bugs, légal, copyright…)

DRY principle

Don't Repeat Yourself

```java
public int score() {
    int involved = getIntegerValue(involvedCards);
    return kind.getMagnitude() + involved;
}

@Override public int compareTo(Combination that) {
    Integer thisScore = this.score();
    Integer thatScore = that.score();
    int raw = thisScore.compareTo(thatScore);
    if (raw != 0) { // Different combination !
        return raw;
    } else {        // Same Combination => using remaining cards
        thisScore = getIntegerValue(remainingCards);
        thatScore = getIntegerValue(that.remainingCards);
        return thisScore.compareTo(thatScore);
    }
}

private int getIntegerValue(Collection<Card> cards) {
    Card[] sorted = cards.toArray(new Card[cards.size()]);
    Arrays.sort(sorted);
    int result = 0;
    int powerOfTen = 1;
    for(int i = 0; i < sorted.length; i++) {
        Card current = sorted[i];
        result += current.getFace().getValue() * powerOfTen;
        powerOfTen *= 10;
    }
    return result;
}
```

DRY!

https://github.com/ace-lectures/H20-MGL-7460/blob/master/cours/9_cc_tests.pdf

KISS : Keep It Simple Stupid

A simple solution is better than a complex one,
even if the solution looks stupid.

How will I benefit from KISS

- You will be able to solve more problems, faster.
- You will be able to produce code to solve complex problems in fewer lines of code
- You will be able to produce higher quality code
- You will be able to build larger systems, easier to maintain
- You're code base will be more flexible, easier to extend, modify or refactor when new requirements arrive
- You will be able to achieve more than you ever imagined
- You will be able to work in large development groups and large projects since all the code is stupid simple

http://people.apache.org/~fhanik/kiss.html

Boy Scout Rule

Leave the Campground cleaner than you found it.

C’est aussi un dépôt de code propre !

- **Ce que l’on a envie de voir dans un dépôt de code (e.g., open source) :**
  - Un fichier **README** bien formaté, qui explique ce que le projet fait et pourquoi.
  - Un fichier ou une section **Getting Started** avec les instructions pour faire tourner le projet.
  - Un fichier de **licence**.
  - Un fichier **How to contribute**, expliquant comment contribuer à votre projet.
  - Une **documentation** technique.
  - Le **code source**, bien structuré.
  - De quoi **construire** votre projet (et le déployer).

- **Ce que l’on ne veut pas voir dans un dépôt de code :**
  - Les exécutables, ce que vous construisez !
  - Des fichiers spécifiques à votre OS (.DS_STORE), à votre IDE, etc.
  - Des données sensibles.
  - Tout ce qui n’est pas lié à votre code.